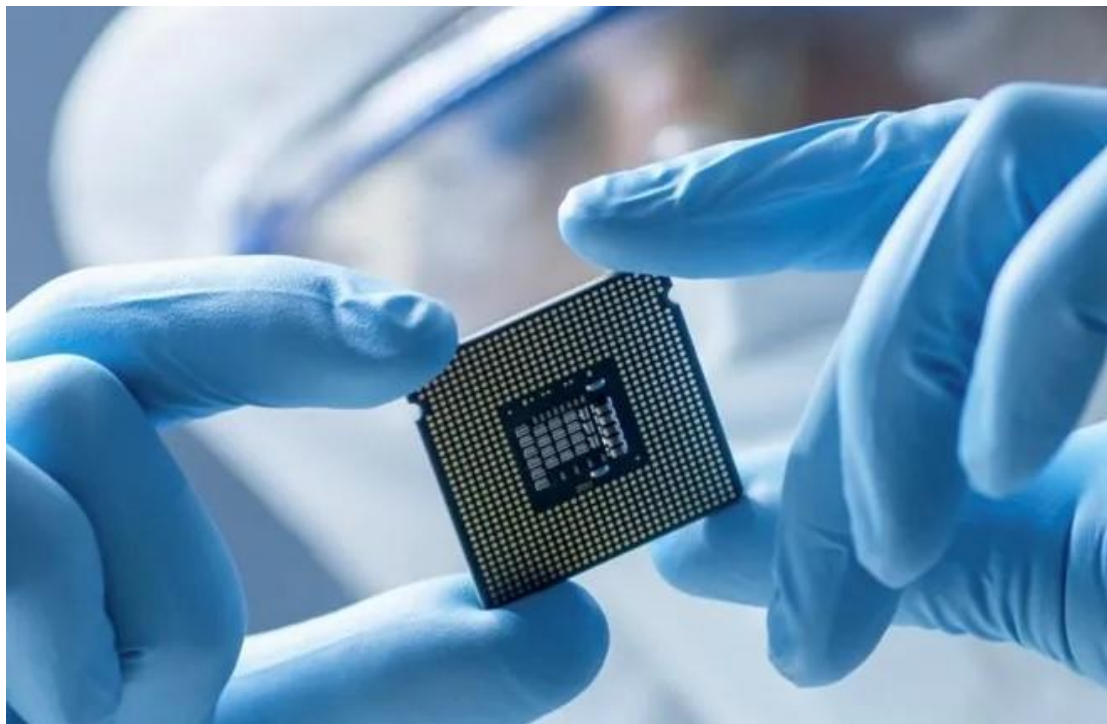




軟硬體協同設計

HW3

林承羿 | 01257027

基礎題

*在 OLED 上顯示自己的學號+姓名(尺寸 128*30bits，學號在上、姓名在下)，字體越大越高分，

*加分題：動畫效果，學號往右跑，姓名往左跑

程式碼截圖與說明

```
13 // student-id
14 unsigned char Bmp001[] = {
15 /*-----
16 ; If font display distortion, please check Fonts format of setup.
17 ; Source file / text :01257027
18 ; Width x Height (pixels) :128X32
19 ; Font Format/Size : Monochrome LCD Fonts ,Vertical scan , Little endian order/512Byte
20 ; Font make date : 2026/3/19 ?? 05:12:02
21 -----*/
22 //Width pixels,Height pixels,Width bytes
23 0x00,0x00,0x00,0x00,0x00,0x00,0x80,0x80,0x80,0x80,0x00,0x00,0x00,0x00,0x00,0x00,
24 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x80,0x80,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
25 0x00,0x00,0x00,0x00,0x00,0x80,0x80,0x80,0x80,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
26 0x00,0x00,0x00,0x00,0x00,0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x00,0x00,0x00,
27 0x00,0x00,0x00,0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x00,
28 0x00,0x00,0x00,0x00,0x00,0x00,0x80,0x80,0x80,0x80,0x00,0x00,0x00,0x00,0x00,0x00,
29 0x00,0x00,0x00,0x00,0x00,0x80,0x80,0x80,0x80,0x80,0x00,0x00,0x00,0x00,0x00,0x00,
30 0x00,0x00,0x00,0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x80,0x00,
31 0x00,0xE0,0xF8,0xFE,0x0F,0x01,0x00,0x00,0x00,0x01,0x0F,0xFE,0xF8,0xE0,0x00,0x00,
32 0x00,0x00,0x00,0x02,0x03,0x03,0xFF,0xFF,0xFF,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
33 0x00,0x18,0x06,0x03,0x01,0x01,0x01,0x01,0x03,0x07,0xFF,0xFE,0x3C,0x00,0x00,0x00,
34 0x00,0x00,0x40,0x70,0x7C,0x77,0xF3,0xE3,0xE3,0xE3,0xC3,0x83,0x00,0x00,0x00,0x00,
35 0x00,0x18,0x07,0x03,0x03,0x03,0x03,0x03,0x03,0xC3,0xFB,0x7F,0x0F,0x01,0x00,0x00,
36 0x00,0xE0,0xF8,0xFE,0x0F,0x01,0x00,0x00,0x00,0x01,0x0F,0xFE,0xF8,0xE0,0x00,0x00,
37 0x00,0x18,0x06,0x03,0x01,0x01,0x01,0x01,0x03,0x07,0xFF,0xFE,0x3C,0x00,0x00,0x00,
38 0x00,0x18,0x07,0x03,0x03,0x03,0x03,0x03,0x03,0xC3,0xFB,0x7F,0x0F,0x01,0x00,0x00,
39 0x00,0x3F,0xFF,0xFF,0x80,0x00,0x00,0x00,0x00,0x80,0xFF,0xFF,0x3F,0x00,0x00,0x00,
40 0x00,0x00,0x00,0x00,0x00,0x00,0xFF,0xFF,0xFF,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
41 0x00,0x00,0x00,0x00,0x00,0x80,0xC0,0x60,0x10,0x0C,0x07,0x03,0x00,0x00,0x00,0x00,
42 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x01,0x03,0x07,0xFF,0x7E,0x00,0x00,0x00,0x00,
43 0x00,0x00,0x00,0x00,0x00,0x00,0x80,0xF0,0xFE,0x1F,0x03,0x00,0x00,0x00,0x00,0x00,
44 0x00,0x3F,0xFF,0xFF,0x80,0x00,0x00,0x00,0x00,0x80,0xFF,0xFF,0x3F,0x00,0x00,0x00,
45 0x00,0x00,0x00,0x00,0x00,0x80,0xC0,0x60,0x10,0x0C,0x07,0x03,0x00,0x00,0x00,0x00,
46 0x00,0x00,0x00,0x00,0x00,0x00,0x80,0xF0,0xFE,0x1F,0x03,0x00,0x00,0x00,0x00,0x00,
47 0x00,0x00,0x00,0x03,0x07,0x0C,0x08,0x08,0x08,0x04,0x07,0x03,0x00,0x00,0x00,0x00,
48 0x00,0x00,0x00,0x08,0x08,0x08,0x0F,0x0F,0x0F,0x08,0x08,0x08,0x00,0x00,0x00,0x00,
49 0x00,0x08,0x0C,0x0E,0x0F,0x0D,0x0C,0x0C,0x0C,0x0C,0x0C,0x0C,0x0E,0x03,0x00,0x00,
50 0x00,0x00,0x06,0x0E,0x0C,0x0C,0x0C,0x04,0x06,0x03,0x01,0x00,0x00,0x00,0x00,0x00,
51 0x00,0x00,0x00,0x00,0x00,0x00,0x0C,0x0F,0x07,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
52 0x00,0x00,0x00,0x03,0x07,0x0C,0x08,0x08,0x04,0x07,0x03,0x00,0x00,0x00,0x00,0x00,
53 0x00,0x08,0x0C,0x0E,0x0F,0x0D,0x0C,0x0C,0x0C,0x0C,0x0C,0x0C,0x0E,0x03,0x00,0x00,
54 0x00,0x00,0x00,0x00,0x00,0x00,0x0C,0x0F,0x07,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
55 };
```

```

57 // chinese name
58 unsigned char Bmp002[] = {
59 /*-----
60 ; If font display distortion, please check Fonts format of setup.
61 ; Source file / text :林承羿
62 ; Width x Height (pixels) :128X32
63 ; Font Format/Size : Monochrome LCD Fonts ,Vertical scan , Little endian order/512Byte
64 ; Font make date : 2026/3/19 ?? 05:21:12
65 -----*/
66 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xC0,0xC0,0xC0,0xC0,0xC0,0xC0,0xFF,
67 0xFF,0xC0,0xC0,0xC0,0xC0,0xC0,0xC0,0xC0,0xC0,0xC0,0xC0,0xC0,0xFF,0xFF,0xC0,
68 0xC0,0xC0,0xC0,0xC0,0xC0,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
69 0x00,0x06,0x06,0x06,0x06,0x06,0x06,0x06,0x06,0x06,0x86,0xC6,0xC6,0x66,0x76,
70 0x36,0x1E,0x0E,0x0E,0x00,0x00,0x00,0x00,0x80,0x80,0x00,0x00,0x00,0x00,0x06,
71 0x26,0x36,0x66,0x66,0xC6,0xC6,0x86,0x06,0x06,0x06,0x06,0xFE,0xFE,0x00,0x00,0x06,
72 0x26,0x36,0x66,0xE6,0xC6,0x86,0x06,0x06,0x06,0x06,0xFE,0xFE,0x00,0x00,0x00,0x00,
73 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
74 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x80,0xE0,0x78,0x1E,0xFF,
75 0xFF,0x1C,0x30,0xE0,0x80,0x00,0x00,0x00,0x00,0xC0,0x70,0x1E,0xFF,0xFF,0x1E,
76 0xF0,0xC0,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x06,0x06,0x06,0x06,
77 0x86,0xFE,0x7E,0x00,0x00,0x18,0x18,0x18,0x18,0xFF,0xFF,0x19,0x18,0x18,0x18,
78 0x00,0x00,0xF0,0xF8,0x1C,0x0C,0x06,0x03,0x01,0x00,0x00,0x00,0x00,0x00,0x40,
79 0xC0,0x60,0x60,0x30,0x18,0x19,0x0C,0x66,0x66,0x62,0x60,0x7F,0x3F,0x00,0x20,0x60,
80 0x30,0x30,0x18,0x18,0x0C,0x0C,0x06,0x66,0x63,0x60,0x7F,0x3F,0x00,0x00,0x00,0x00,
81 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
82 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x10,0x1C,0x0E,0x03,0x01,0x00,0x00,0xFF,
83 0xFF,0x00,0x00,0x00,0x01,0x83,0xE0,0x30,0x1C,0x07,0x01,0x00,0x00,0xFF,0xFF,0x00,
84 0x00,0x03,0x07,0x1C,0x30,0xE0,0x80,0x00,0x00,0x00,0x00,0x00,0x00,0xC0,0xF8,
85 0x3F,0x0F,0x00,0xC0,0xC3,0xC3,0xC3,0xC3,0xC3,0xFF,0xC3,0xC3,0xC3,0xC3,
86 0xC3,0xC0,0x00,0x0F,0x3E,0xF0,0xC0,0x00,0x00,0x00,0x00,0x00,0x18,0x18,
87 0x18,0x18,0x18,0x18,0x18,0xFF,0xFF,0x18,0x18,0x18,0x18,0x18,0x18,0x18,
88 0x18,0x18,0xFF,0xFF,0x18,0x18,0x18,0x18,0x18,0x18,0x00,0x00,0x00,
89 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
90 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x7F,
91 0x7F,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x7F,0x7F,0x00,
92 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x04,0x0E,0x07,0x03,0x00,
93 0x00,0x00,0x00,0x00,0x00,0x60,0x60,0x60,0x60,0x7F,0x3F,0x00,0x00,0x00,
94 0x00,0x00,0x00,0x00,0x00,0x03,0x07,0x0E,0x04,0x00,0x00,0x00,0x00,0x20,
95 0x60,0x30,0x30,0x18,0x18,0x0C,0x0F,0x03,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
96 0x00,0x00,0x00,0x7F,0x7F,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
97 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
98 };

```

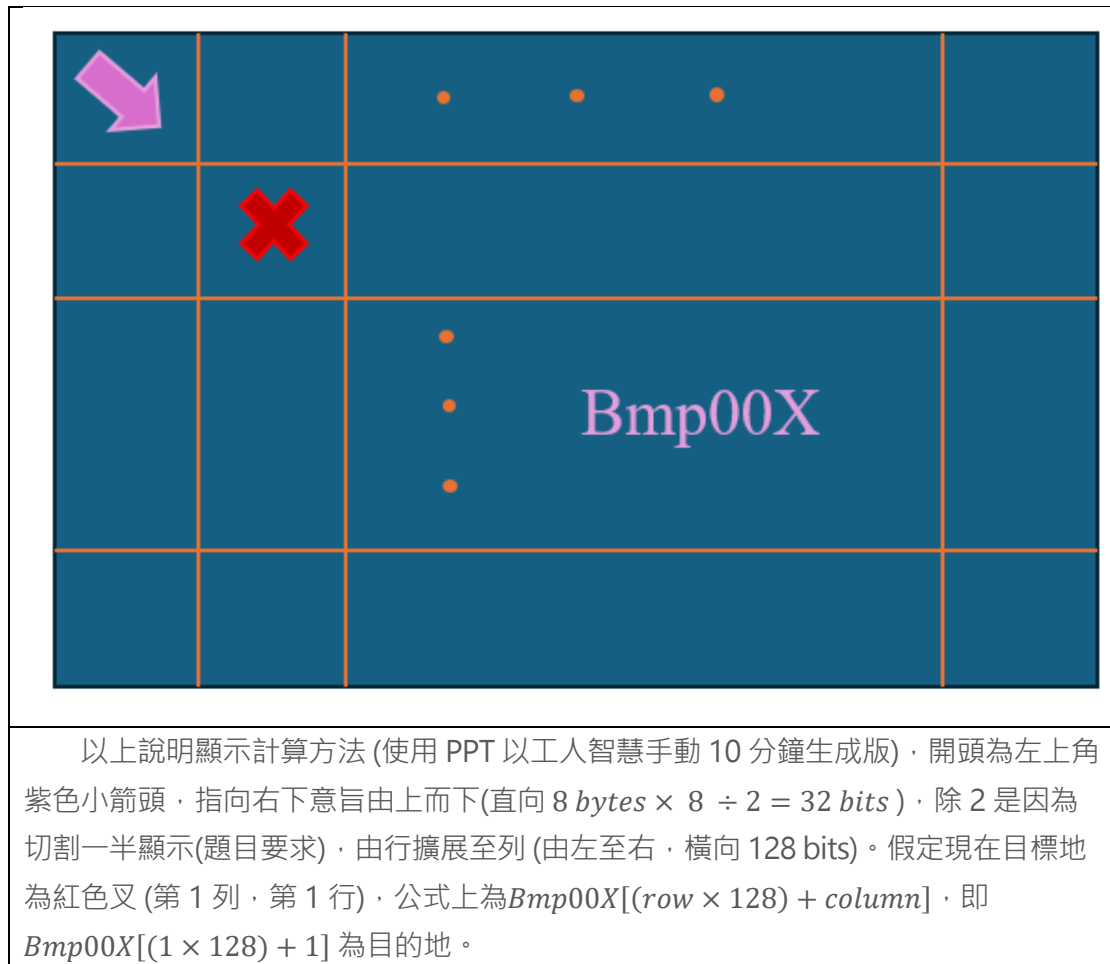
雖然你不會看，我也不想看，但證明這兩大塊醜東西可以在 OLED 上顯示本人大名、學號。特別說明，經過不上 10 種字型的測試，發現 Times New Roman 可以讓學號在字形 24 充滿 OLED 空間。中文使用微軟正黑體可以保證在 OLED 可接受最大字型仍不會切到上下空間。

```

115 void OLED_Bmp001()
116 {
117     u16 i,j;
118
119     for(i=0;i<=3;i++)
120     {
121         OLED_Set_Pos(0,i);
122         for(j=0;j<128;j++)
123         {
124             OLED_WR_Byte(Bmp001[i*128+j],OLED_DATA);
125         }
126     }
127
128 }
129
130 void OLED_Bmp002()
131 {
132     u16 i,j;
133
134     for(i=0;i<=3;i++)
135     {
136         OLED_Set_Pos(0,i+4);
137         for(j=0;j<128;j++)
138         {
139             OLED_WR_Byte(Bmp002[i*128+j],OLED_DATA);
140         }
141     }
142
143 }

```

這就是基礎題的顯示方式(加分題也大同小異)，如果是自己寫肯定不會分兩個，應該是參數塞 array，(i+?)的?應該使用變數，決定是否+4 (OLED 列數從哪開始)。但基於老師都已經給了，copy and paste 可是懶惰鬼的最高指導，我肯定是能省則省 (連排版都懶)。




```

146 int main(void)
147 {
148     u8 key;
149     u8 datatemp[SIZE];
150     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //設置系統中斷優先級分組2
151     delay_init(168); //初始化延時函數
152     uart_init(115200); //初始化串口波特率為115200
153
154     LED_Init(); //初始化LED
155     LCD_Init(); //LCD初始化
156     KEY_Init(); //按鍵初始化
157     AT24CXX_Init(); //IIC初始化
158     IIC1_Init();
159     OLED_Init();
160     LCD_Clear(BLACK);
161     POINT_COLOR=RED;
162     LCD_ShowString(30,50,200,16,16,"Explorer STM32F4");
163     LCD_ShowString(30,70,200,16,16,"IIC TEST");
164     LCD_ShowString(30,90,200,16,16,"ATOM@ALIENTEK");
165     LCD_ShowString(30,110,200,16,16,"2014/5/6");
166     LCD_ShowString(30,130,200,16,16,"KEY1:Write KEY0:Read"); //顯示提示信息
167     while(AT24CXX_Check())//檢測不到24c02
168     {
169         LCD_ShowString(30,150,200,16,16,"24C02 Check Failed!");
170         delay_ms(500);
171         LCD_ShowString(30,150,200,16,16,"Please Check! ");
172         delay_ms(500);
173         LED0=!LED0;//DS0閃爍
174     }
175     LCD_ShowString(30,150,200,16,16,"24C02 Ready!");
176     POINT_COLOR=BLUE; //設置字體為藍色
177
178     OLED_Bmp001();
179     OLED_Bmp002();
180     while(1)
181     {
182         key=KEY_Scan(0);
183         if(key==KEY1_PRES)//KEY1按下,寫入24C02
184         {
185             LCD_Fill(0,170,239,319,BLACK); //清除半屏
186             LCD_ShowString(30,170,200,16,16,"Start Write 24C02....");
187             AT24CXX_Write(0,(u8*)TEXT_Buffer,SIZE);
188             LCD_ShowString(30,170,200,16,16,"24C02 Write Finished!"); //提示傳送完成
189         }
190         if(key==KEY0_PRES)//KEY0按下,讀取字符串並顯示
191         {
192             LCD_ShowString(30,170,200,16,16,"Start Read 24C02.... ");
193             AT24CXX_Read(0,datatemp,SIZE);
194             LCD_ShowString(30,170,200,16,16,"The Data Readed Is: "); //提示傳送完成
195             LCD_ShowString(30,190,200,16,16,datatemp); //顯示讀到的字符串
196         }
197         delay_ms(10);
198     }
199 }

```

其實重點只有 178、179 行，其他只是懶得刪掉，反正多了不會影響啥，唯一做出的改動只有 LCD_CLEAR(BLACK)，相信你也知悉盯著白色背景的痛苦，我非常愛護自己的狗眼。

結果說明

結果呈現上如 normal.mov 所示，根據 main 中的邏輯，其實就是只刷新一次，while 裡面的都是廢話，不是很重要。單純藉由兩個 function 將高度切一半，跑完兩個 array 而已。就是一個樸實無華的學號、名字顯示。

加分題

*在 OLED 上顯示自己的學號+姓名(尺寸 128*30bits，學號在上、姓名在下)，字體越大越高分，動畫效果，學號往右跑，姓名往左跑。

程式碼截圖與說明

```
USER > C main.c > Bmp002
1  #include "sys.h"
2  #include "delay.h"
3  #include "usart.h"
4  #include "led.h"
5  #include "lcd.h"
6  #include "24cxx.h"
7  #include "key.h"
8  #include "myiic.h"
9  #include "timer.h"
10 #include "exti.h"
11 #include "stdbool.h"
```

這是經過一個寫 STM32 爆出 RAM 不夠的教訓，第 11 行是個身為細節狗都應該知道的好東西，如何讓 C 真正的有 bool 可以實現 1bit 空間還得是他。

```

99
100  u16 len = 128, height = 4;
101  bool stopContinue = 0, stdIdFg = 1, stdNameFg = 0, lrOrUd = 0;
102
103  void OLED_Bmp001(void);
104  void OLED_Bmp002(void);
105  void shiftBlock(unsigned char*, bool);
106

```

以下說明變數與 function 作用，聲明本人是一個小駝峰命名喜好者，有底線單純是模板搬來不想改。

OLED_Bmp001	顯示大名
OLED_Bmp002	顯示學號
shiftBlock	以 bool 接受 unsigned char[] 應該向左向右、向上向下搬移

len	OLED 橫向長度
height	OLED 直向高度(bytes)
stopContinue	以 bool 使 shiftBlock 知悉是否需要繼續搬移
stdIdFg	告知 shiftBlock 學號搬移方向
stdNameFg	告知 shiftBlock 名字搬移方向
lrOrUd	切換左右搬移、上下搬移模式的 flag

每個變數都是深思熟慮出來的結果，本人在加分題(近乎 99%都是自己寫，1%模板搬運)的情況下希望自己掌控度高(超級控制狂)，不浪費空間亂令變數、函式。

Len、height 是必要的，如果每個固定長數都 hard-code，長寬改變會影響到很多地方，會因為某幾個躲在小角落的鱗東西陰你一手，更好的寫法應該要令一個「xxx.h」，但這只是小作業，意思點到就好，我有點懶。

其餘 bool 都是 flag，皆為了 shiftBlock 服務，計畫以 button 控制每個 flag 的切換促使 OLED 上名字、學號搬移方向發生改變。而因為 shiftBlock 的參數設定沒考慮好(相當於前人留下的東西不想改)，只能上下 or 左右，因此需要 lrOrUd 綁定函式該左右搬還是上下搬。


```

144 // for chinese name (to left)
145 void OLED_Bmp001() {
146     u16 i,j;
147     for(i=0; i<height; i++) {
148         OLED_Set_Pos(0,i);
149         for(j=0;j<len;j++) {
150             OLED_WR_Byte(Bmp001[i*len+j],OLED_DATA);
151         }
152     }
153 }
154
155 // for student-id (to right)
156 void OLED_Bmp002() {
157     u16 i,j;
158     for(i=0; i<height; i++) {
159         OLED_Set_Pos(0,i+height);
160         for(j=0;j<len;j++) {
161             OLED_WR_Byte(Bmp002[i*len+j],OLED_DATA);
162         }
163     }
164 }

```

以上為顯示兩大區塊的函示，跟基本題 87%像，只有將固定常數取代，如基礎題所談論的，應該可藉由傳入參數方式將兩個函式合併，提高代碼的維護性。這時可以看到註解上只有左右，天真的孩子還沒想到之後會加上下方向：P

```

166 // mode 0: left, 1: right
167 void shiftBlock(unsigned char* sor, bool mode) {
168     u16 i, col, offset, row;
169     unsigned char tmp;
170     // adjust left-right
171     if (lrOrUd == 0) {
172         for (i = 0; i < height; i++) {
173             // get begin of row
174             offset = i * len;
175             // shift left
176             if (mode == 0) {
177                 tmp = sor[offset];
178                 // cover from next one
179                 for (col = 0; col < (len-1); col++) {
180                     sor[offset + col] = sor[offset + col + 1];
181                 }
182                 // set the last one
183                 sor[offset + (len-1)] = tmp;
184             } else if (mode == 1) {
185                 // shift right
186                 // get last one
187                 tmp = sor[offset + (len-1)];
188                 // cover from back one
189                 for (col = (len-1); col > 0; col--) {
190                     sor[offset + col] = sor[offset + col - 1];
191                 }
192                 // set the first one
193                 sor[offset] = tmp;
194             }
195         }
196     } else if (lrOrUd == 1) {
197         // adjust up-down
198         for(i = 0; i < len; i++) {
199             // shift up
200             if (mode == 0) {
201                 tmp = sor[i];
202                 for(row=0; row<(height-1); row++) {
203                     sor[(len*row) + i] = sor[(len*(row+1)) + i];
204                 }
205                 sor[i + (len*(height-1))] = tmp;
206             } else if (mode == 1) {
207                 // shift down
208                 tmp = sor[i + (len*(height-1))];
209                 for(row=(height-1); row>0; row--) {
210                     sor[(len*row) + i] = sor[(len*(row-1)) + i];
211                 }
212                 sor[i] = tmp;
213             }
214         }
215     }
216 }
217 return;
218 }

```

如同先前提到的，這函式有先天不良卻領不到殘障手冊的問題，更準確地說，夢想是一步一步達成的，要求只有左右，只有一個 bool 維持左右方向是很合理的。而後 0x1(本人大名)這 PM 開出了要上下移動的要求，直接參數多加一個 bool 也行，但當下 0x1 這 RD 覺得這寫法會有點繁瑣(現在想想其實只是多傳了 IrOrUd 而已)，所以就變成這樣了。

先判斷該左右還是上下，再判斷傳進來的 bool 決定上 or 下，左 or 右。向左搬移即存下最左直行，待每列向左移 1bit 後，回填存下的直行到最右邊行。向右搬移也差不多是這道理，不多做解釋。上下移動即存下最上 or 下列，其餘向上 or 下移動 1bit，最後回填存下的至剩餘列。

```
129 // timer 3 interrupt
130 void TIM3_IRQHandler(void) {
131     if(TIM_GetITStatus(TIM3,TIM_IT_Update)==SET) {
132         if (stopContinue == 0) {
133             OLED_Bmp001();
134             OLED_Bmp002();
135             // shift Bmp001(student-id) to right
136             shiftBlock(Bmp001, stdIdFg);
137             // shift Bmp002(name) to left
138             shiftBlock(Bmp002, stdNameFg);
139         }
140     }
141     TIM_ClearITPendingBit(TIM3,TIM_IT_Update);
142 }
```

講了顯示方式，搬移過程，結合在一起後多久顯示才是問題，本人使用最穩定可控的方式——timer 3 中斷，先判 flag 是否暫停移動，否則重新以指定方向刷新。此過程不需要清除上次顯示內容，與上次作業不同(在 LCD 上畫圖是憑空生成，無法確定上一次畫在哪，暴力的方式就是全部清除。但這次是直接搬移兩個 array，只要完整搬移，結果一定正確，不需整個 OLED 清除)。為了讓顯示加速，我有意切開顯示與搬移這兩塊部分，邊搬邊算會讓邏輯攪在一起，降低可讀、維護性，且邊讀邊算會降低顯示速度(但 OLED 那麼小塊應該感覺不出來)，因此採用先顯示，算完下一次搬移結果才結束中斷(因為先算下一步，才會在影片 2:58 處發生暫停恢復後明明已經切換方向至左右卻還上下移動了一次)。

```

220 // wake-up (0: continue, 1: stop)
221 void EXTI0_IRQHandler(void) {
222     delay_ms(10);
223     if(WK_UP==1) {
224         stopContinue = !stopContinue;
225         // let led0 shine when it stop
226         if (stopContinue == 1) {
227             LED0 = 0;
228         } else LED0 = 1;
229         // this fucking buttton is too old to use easily
230         while(WK_UP == 1);
231     }
232     EXTI_ClearITPendingBit(EXTI_Line0);
233 }
234
235
236 // key-2 adjust (0: left-right, 1: up-down)
237 void EXTI2_IRQHandler(void) {
238     delay_ms(10);
239     if(KEY2==0) {
240         lrOrUd = !lrOrUd;
241     }
242     EXTI_ClearITPendingBit(EXTI_Line2);
243 }
244
245 // key-1 student-name (0: left-up, 1: right-down)
246 void EXTI3_IRQHandler(void) {
247     delay_ms(10);
248     if(KEY1==0) {
249         stdNameFg = !stdNameFg;
250     }
251     EXTI_ClearITPendingBit(EXTI_Line3);
252 }
253
254 // key-0 student-id (0: left-up, 1: right-down)
255 void EXTI4_IRQHandler(void) {
256     delay_ms(10);
257     if(KEY0==0) {
258         stdIdFg = !stdIdFg;
259     }
260     EXTI_ClearITPendingBit(EXTI_Line4);
261 }
262

```

為了達到即刻按壓馬上響應的要求，RD 使用了模板中 key 的外部中斷，但如果不那麼吹毛求疵，寫在 main 中偵測也可 (為了保留 PM 無情增加工作量下的最小修改，讓 main 中 while 先保留空間)。特別留意在 wake_up-key 做了其他事情，這是一件令人鼻酸的故事，這塊 STM32 歷經風霜，在不知道更換幾個主人後被摧殘到令人惋惜的地步，為了確保奴隸(button)是否還正常運作，以 LED0 亮暗告知 RD 是否切換至指定狀態，最後的 while 是因為在以之按鈕不能準確偵測的強況下，RD 採取了長按，為避免重複中斷偵測，應等到放開才脫離中斷。

```
107 int main(void) {
108     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
109     delay_init(168);
110     uart_init(115200);
111
112     LED_Init();
113     LCD_Init();
114     LCD_Clear(BLACK);
115     AT24CXX_Init();
116     IIC1_Init();
117     KEY_Init();
118     OLED_Init();
119     EXTI_Init();
120     // timer 3 is 84M, 84M/8400=10Khz, 500*10Hz=50ms
121     TIM3_Int_Init(500-1,8400-1);
122
123     while(1) {
124
125     }
126     return 0;
127 }
```

可以看到 main 中其實沒啥東西，其實也不是很重要。感謝 PM 對 RD 的同情，考慮開發資金斷鍊(時間不夠)的前題下，無實現其餘功能，想法是有的(加減速)，但沒有實現。需要留意的是每個 init 的必要性，透過上次作業，已知 uart_init 與 LCD 需要綁定，而 LCD 與 AT24CXX_Init 綁定(我沒詳細 view code，但註解後就是不會動)，又確信 OLED 是經由 IIC 傳輸協定規範，因此每個 init 都是必要的。Delay 是拿來解彈跳的，LED 確保 wake-up-key 的運作，exti 為保證四個 key 的即時響應，timer 3 保證顯示刷新時間。

結果說明

顯示結果如 bonus.mov 所示，可以看到每個功能都如 PM 開出的規格一樣，穩妥的運轉著。但某個 key 即使做了解彈跳、引許長按，還是大爆炸，這顆按鍵已經快要歸西了，影片中的它也是非常的任性，需要一聲「ㄍㄝ」(影片 2:40) 才好好工作一回。

心得

這次的作業過程可說是——非常不順利，老師一再提及不要把 VCC 與 GND 接反，我每次都關電後才接線，檢查兩三次後才開電，每次燒 code 結果都不一樣，要點亮本身就是一件極其困難的事情，亮了之後還有大機率是爛掉的。一再排除，三用電表測板子電壓輸出都是正常的，量測 OLED 接收電壓也很正常。直到我換了線終於搞定了。這破線不只 8051 搞我，連 STM32 都還不忘回來陰我一手。

此次作業時間只有兩天，由於開發員想要在每個禮拜日前(一個禮拜的結束)完成當禮拜的所有事情，所以實屬無太多時間可以留心此作業。沒實現的功能為透過 timer 3 設定不同中段時間達到加減速的效果，更仔細地說是一個 u16 的變數初始為 500，透過某個指定動作加減 10，重新啟動 TIM3_Int_Init 的函式。未實現的原因有以下幾點，本人不想有短按、長按的分別(當時 4 個 key 已經有各自功能)。如果要區分長短按，還要透過多開一個 timer 達成，流程中勢必得有一個 while 讓流程卡住，使中斷計數，但問題就在這，我按下 key 已經是發生中斷，在中斷裡寫 while 讓其他 timer 中斷計數？所以要跳開中斷處理其他中斷，需要設定優先級問題才可以辦到(我還沒撞過這大坑，不敢嘗試)，因此大抵上是按鍵不夠、中斷優先級設置的問題。

當然，如果有時間，這些都是可以解決的，開發空間是有的，可以透過 delay 硬是把邏輯塞在 main 中完成，去掉所有中斷(以 main 控制所有 flag，避免中斷發生而脫離長按、短按的計算)，但這樣是捨棄維護性、可讀性的操作，這是種非常暴力的行為。最後的想法是透過 uart 實現，但我不確定 uart 是否是被占用的，表面上看起來是可以透過 baud-rate=115200 傳輸，但 uart 與 LCD 掛勾，不確定 uart 是否已經是為 LCD 服務的情況下，是否可以接收我傳輸的訊息(可透過開啟 uart 2、3 繞開這問題)。如果可以使用 uart，只需要規範最快速度即可(避免中斷發生時間 ≤ 0)，而這是不確定的，本人尚未確定 IIC 傳輸最快限制，太快的中斷也許引發 IIC 傳輸上的錯誤，使 OLED 上顯示出錯。

開發中的小細節即老師提及的「越是塞滿空間，越高分」，花了些時間尋覓字體，最終確信數字字體解答之一即 Times New Roman，中文的字體解答之一為微軟正黑

體。設計如何動這件事情是經過改版的，一開始我還沒認知到要不要先整片清掉再重新畫上新的，第一版是清掉再重畫，它會有舊式映像管電視機的獨特感覺(由上而下慢慢刷下來的那條黑線)，經過評估過後，我確定可以不用先清屏，才變成 demo 影片中所呈現的清晰樣貌。

移動的設計也是經過演變的，一般而言，身為一個程式開發者，一般不會大規模頻繁搬動陣列，這是一個擺明很耗費時間的操作，但這作業還是這樣做了。一開始當然不是這樣的，關於陣列的操作，起手勢應該操作一個可變的 index，以數學運算的方式配合 mod 維護邊界問題，但做完只剩下學號，不久之後全部黑屏。猜測 OLED 顯示規則還有老師沒有講明的，考慮到我其實也沒啥時間再讀 reference、code，採取最暴力的解法，已知可以顯示陣列，那就在顯示之前處理下一刻移動的結果存入陣列就好。這方法成立的大前提是 OLED 不能夠太大，具體大到什麼程度還在可接受範圍仍未知，但大抵上要配合中斷的時間，如果還沒搬移完陣列下一個 timer interrupt 進來就完了，會造成顯示大量的延遲(得虧這 OLED 夠小塊)。

至於開發資金(時間)不夠的原因是專題要燒起來，許教授說如果下次開會前沒搞出東西就當場來段印度 F4(<https://www.youtube.com/watch?v=wjz2c7YKEg0>)，我不是很想。所以我做到高於加分題要求就收手。且不因為趕時間就把東西丟進去 AI 熔爐出一個維護性很差的東西是本人的堅持(感化自 NISSAN-GTR VR38 引擎每個細節堅持人工組裝)，這是對於提交一份作業，對於本人這塊招牌的品質保證。

此次作業特別鳴謝 gemini 聰明的大腦，左右移動一開始我寫的爛掉了，經過 AI 提示過後，我變得更加的原始(暴力)，信奉東瀛戰神理論——跑不過就爆改(引用自 NISSAN-GTR 為人稱頌的改裝戰 <https://www.youtube.com/watch?v=ILPnY0DRyg>)，它成功實現了功能，我依樣畫葫蘆生出了上下移動。感謝新的四條杜邦線贊助，讓 OLED 每次顯示的結果都是固定的，每次開機結果都不一樣真的是堪比薛丁格的貓。